

Cuarentena
Día
145
03/08/2020

TD1

*Esperando que se conecten
Al ZOOM*

Bienvenidos
Año 2020



Nuevas directivas

Para entrar al ZOOM.2020

Deben ser usuario registrado de ZOOM

Deben figurar: *Curso Legajo Apellido y nombre*

No cumplir, deberé eliminarlo y no podrán volver a entrar a la sección.

Espero no ofender a nadie, pero es por seguridad de todos

- Verilog
 - Circuitos combinacionales básicos.
 - Registros
 - buses
 - Resolver problemas con Decodificadores, Multiplexores, Demultiplexores,,Codificadores con Verilog.
 - Funciones con multiplexores - Sincronos y combinacionales

- Verilog
- Registros
- buses
- clk

```
module RegyComb(  
    input clock,  
    input a,  
    input b,  
    input c,  
    output sal,  
    output sal2,  
    output reg Rsal  
);
```

```
// assign sal=a & b;
```

```
assign sal2 = Rsal | b;
```

```
always @(posedge clock )
```

```
Rsal = c | b;
```

```
assign sal=c | b;
```

```
// assign sal=a & b;
```

```
endmodule
```

Combinatorio

registro

- Verilog
 - Registros
 - buses
 - clk

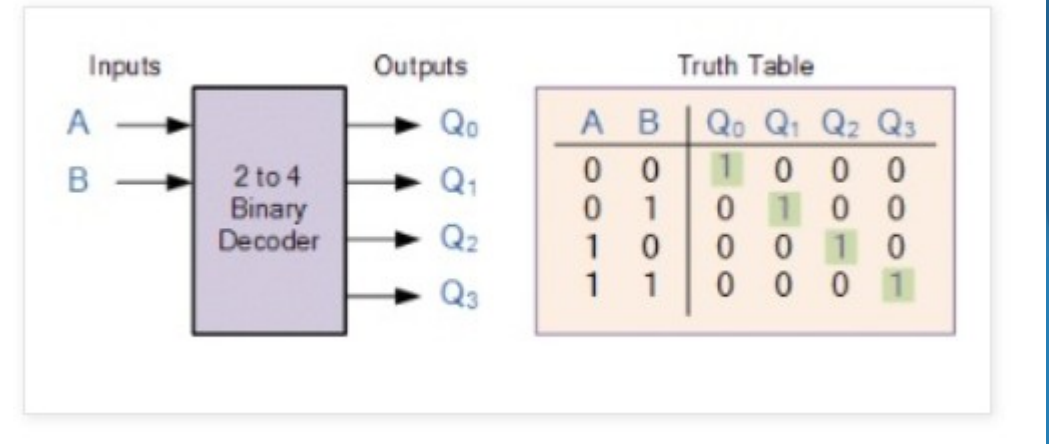
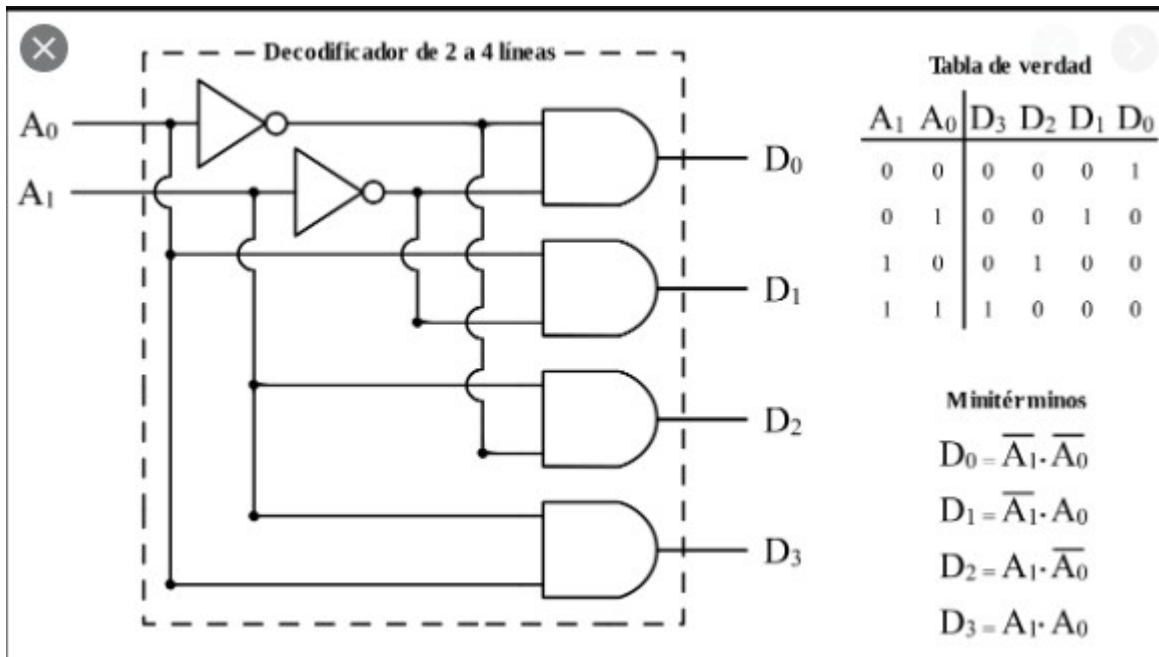
Verilog con Xilinx

- RegyComb
- RegyComb2

Unidad Temática N° 5: Circuitos combinacionales. (Cap. 4)

TD1

- Verilog
- Resolver problemas con Decodificadores



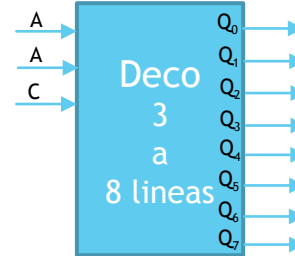
Unidad Temática N° 5: Circuitos combinacionales. (Cap. 4)

TD1

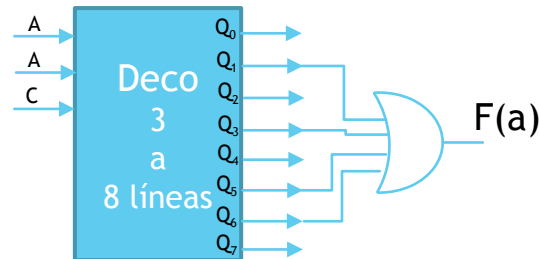
➤ Verilog

➤ Resolver problemas con Decodificadores

	A	B	C	Q(0)	Q(1)	Q(2)	Q(3)	Q(4)	Q(5)	Q(6)	Q(7)
m ₀	0	0	0	1	0	0	0	0	0	0	0
m ₁	0	0	1	0	1	0	0	0	0	0	0
m ₂	0	1	0	0	0	1	0	0	0	0	0
m ₃	0	1	1	0	0	0	1	0	0	0	0
m ₄	1	0	0	0	0	0	0	1	0	0	0
m ₅	1	0	1	0	0	0	0	0	1	0	0
m ₆	1	1	0	0	0	0	0	0	0	1	0
m ₇	1	1	1	0	0	0	0	0	0	0	1



	A	B	C	F(a)
m ₀	0	0	0	0
m ₁	0	0	1	1
m ₂	0	1	0	0
m ₃	0	1	1	1
m ₄	0	0	0	0
m ₅	0	0	1	1
m ₆	0	1	0	1
m ₇	0	1	1	0



Decodificaror para generar funciones

	A	B	F(a)
m ₀	0	0	1
m ₁	0	1	0
m ₂	1	0	1
m ₃	1	1	0

```
module Deco_fun(  
    input clk,  
    input reset,  
    input [1:0] A,  
    output Sal_C,  
    output reg [3:0] Sal  
);
```

```
always @(posedge clk)  
    if (reset)  
        Sal <= 4'h0;    // hexadecimal  
    else  
        case (A)  
            2'b00 : Sal <= 4'b0001;  
            2'b01 : Sal <= 4'b0010;  
            2'b10 : Sal <= 4'b0100;  
            2'b11 : Sal <= 4'b1000;  
            default : Sal <= 4'b0000;  
        endcase
```

Reset síncrono

assign Sal_C= Sal[0] | Sal[2];

En
Verilog

Decodificaror para generar funciones

	A	B	F(a)
m ₀	0	0	1
m ₁	0	1	0
m ₂	1	0	1
m ₃	1	1	0

```
module Deco_fun(  
    input clk,  
    input reset,  
    input [1:0] A,  
    output Sal_C,  
    output reg [3:0] Sal  
);
```

```
always @(posedge clk , posedge reset)  
    if (reset)  
        Sal <= 4'h0;    // hexadecimal  
    else  
        case (A)  
            2'b00 : Sal <= 4'b0001;  
            2'b01 : Sal <= 4'b0010;  
            2'b10 : Sal <= 4'b0100;  
            2'b11 : Sal <= 4'b1000;  
            default : Sal <= 4'b0000;  
        endcase
```

Reset Asincrono

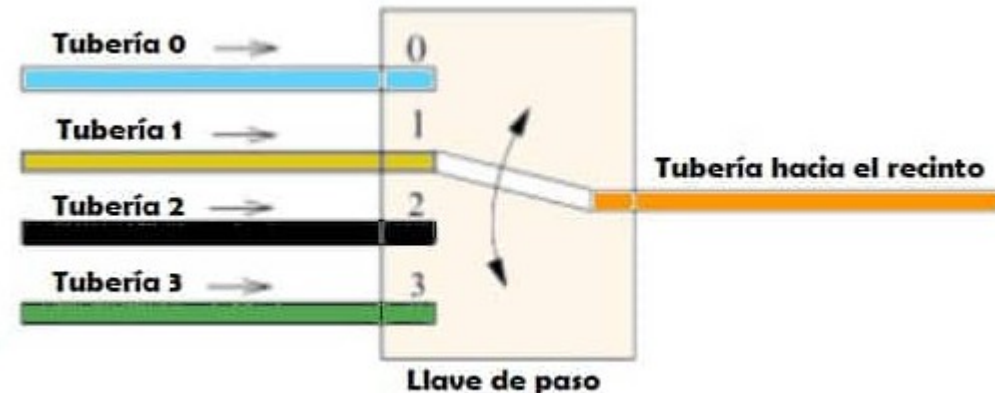
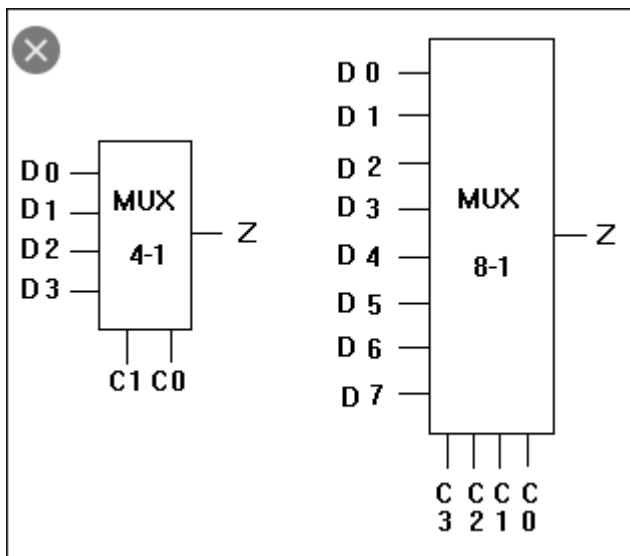
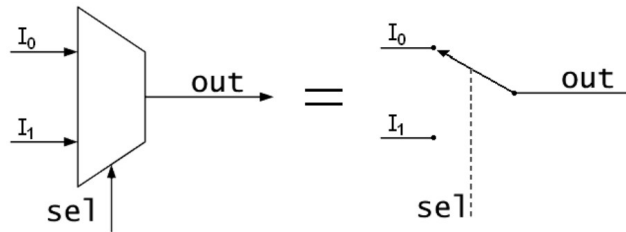
assign Sal_C= Sal[0] | Sal[2];

En
Verilog

Unidad Temática N° 5: Circuitos combinacionales. (Cap. 4)

TD1

- Verilog
- Resolver problemas con multiplexores



Verilog

Resolver problemas con multiplexores

	A	B	C	F(a)
m ₀	0	0	0	0
m ₁	0	0	1	1
m ₂	0	1	0	0
m ₃	0	1	1	1
m ₄	1	0	0	1
m ₅	1	0	1	0
m ₆	1	1	0	0
m ₇	1	1	1	0

wire [2:0] select;

assign select[0]=C;
assign select[1]=B;
assign select[2]=A;

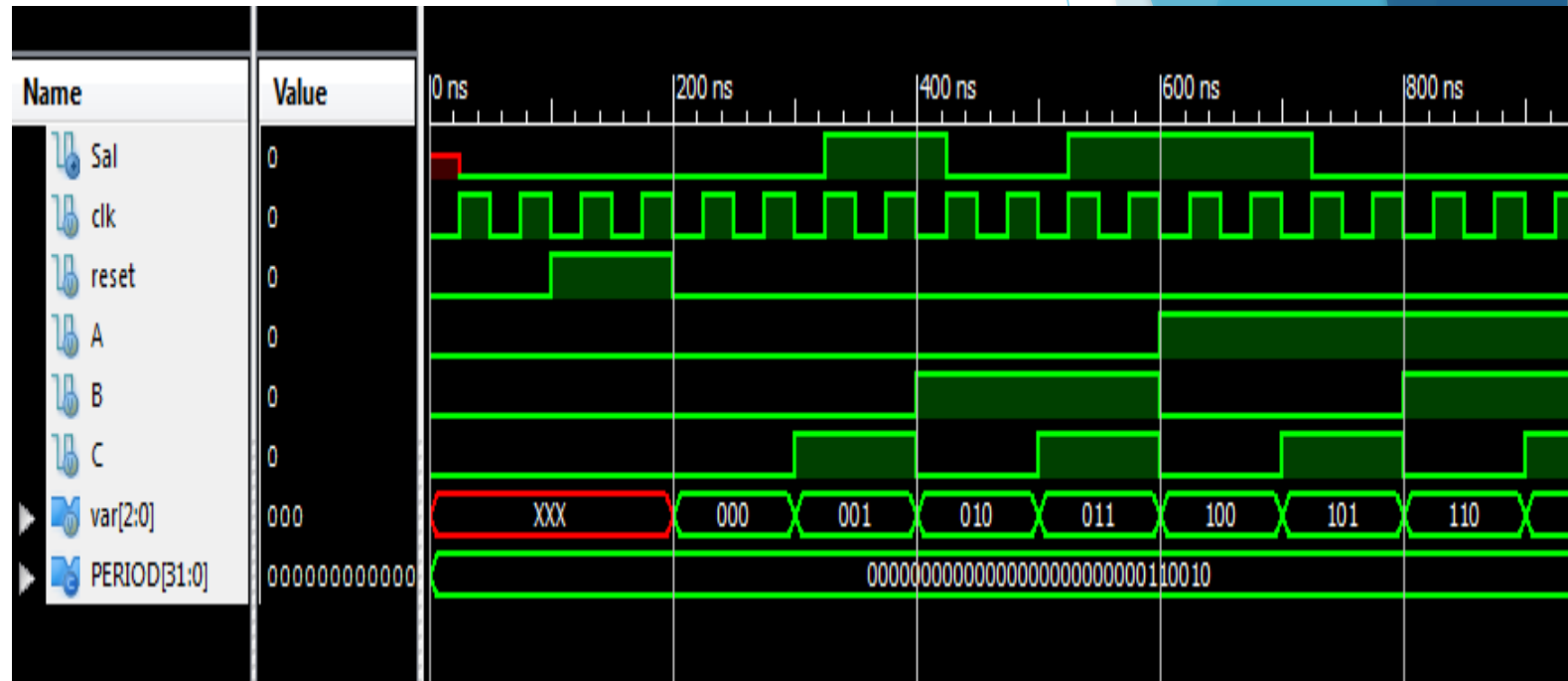
always @(posedge clk)

```
case (select)
  3'b000: Sal = 0;
  3'b001: Sal = 1;
  3'b010: Sal = 0;
  3'b011: Sal = 1;
```

```
  3'b100: Sal = 1;
  3'b101: Sal = 0;
  3'b110: Sal = 0;
  3'b111: Sal = 0;
```

endcase

```
module
Mult_fun(
  input clk,
  input reset,
  input A,
  input B,
  input C,
  output reg Sal
);
```



4-35. Implementar las siguientes funciones booleanas con multiplexores 4 a 1 y compuertas.

(a) $F_1(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$

(b) $F_2(A, B, C, D) = \Sigma(1, 2, 5, 7, 8, 10, 11, 13, 15)$

	A	B	C	D	F(a)	
m ₀	0	0	0	0	0	0
m ₁	0	0	0	1	1	1
m ₂	0	0	1	0	0	1
m ₃	0	0	1	1	1	0
m ₄	0	1	0	0	1	0
m ₅	0	1	0	1	0	1
m ₆	0	1	1	0	0	0
m ₇	0	1	1	1	0	1
m ₈	1	0	0	0	0	1
m ₉	1	0	0	1	0	0
m ₁₀	1	0	1	0	0	1
m ₁₁	1	0	1	1	1	1
m ₁₂	1	1	0	0	1	0
m ₁₃	1	1	0	1	1	1
m ₁₄	1	1	1	0	1	0
m ₁₅	1	1	1	1	1	1

D	C=0	C=1
0	0	1
1	0	1

$F = D$

D	C=0	C=1
0	1	0
1	0	0

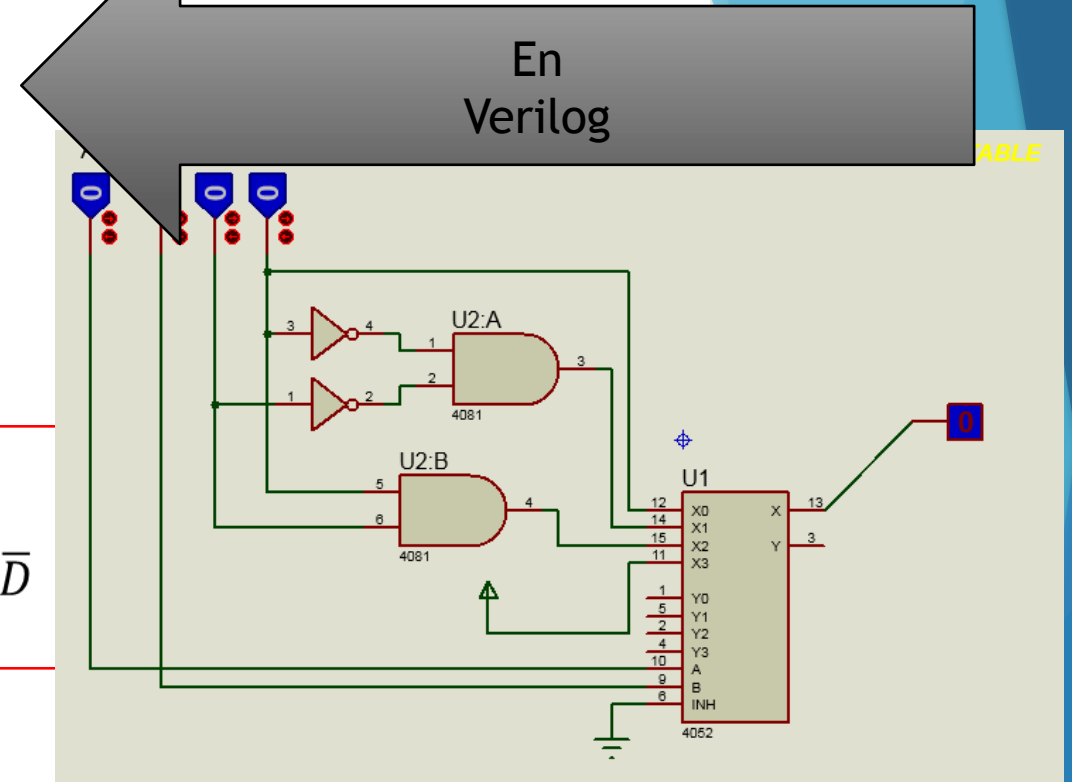
$F = \bar{C} \cdot \bar{D}$

D	C=0	C=1
0	0	0
1	0	1

$F = C \cdot D$

D	C=0	C=1
0	1	1
1	1	1

$F = 1$



Simular

4-35. Implementar las siguientes funciones booleanas con multiplexores 4 a 1 y compuertas.

(a) $F_1(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$

(b) $F_2(A, B, C, D) = \Sigma(1, 2, 5, 7, 8, 10, 11, 13, 15)$

	A	B	C	D	F(a)	
m ₀	0	0	0	0	0	0
m ₁	0	0	0	1	1	1
m ₂	0	0	1	0	0	1
m ₃	0	0	1	1	1	0
m ₄	0	1	0	0	1	0
m ₅	0	1	0	1	0	1
m ₆	0	1	1	0	0	0
m ₇	0	1	1	1	0	1
m ₈	1	0	0	0	0	1
m ₉	1	0	0	1	0	0
m ₁₀	1	0	1	0	0	1
m ₁₁	1	0	1	1	1	1
m ₁₂	1	1	0	0	1	0
m ₁₃	1	1	0	1	1	1
m ₁₄	1	1	1	0	1	0
m ₁₅	1	1	1	1	1	1

	D		
	0	1	
C	0	0	1
	1	0	1

 $F = D$

	D		
	0	1	
C	0	1	0
	1	0	0

 $F = \bar{C} \cdot \bar{D}$

	D		
	0	1	
C	0	0	0
	1	0	1

 $F = C \cdot D$

	D		
	0	1	
C	0	1	1
	1	1	1

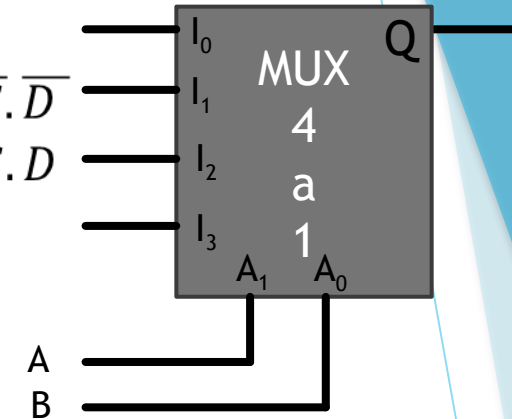
 $F = 1$
En
Verilog//Clase 11 circuitos combinacionales con
mux // //(pag 12)assign select_2[0]=B;
assign select_2[1]=A;always @(posedge clk)
case (select_2)
2'b00: Sal_2 = ~D;
2'b01: Sal_2 = (~C & ~D);
2'b10: Sal_2 = (C & D);
2'b11: Sal_2 = 1;

endcase

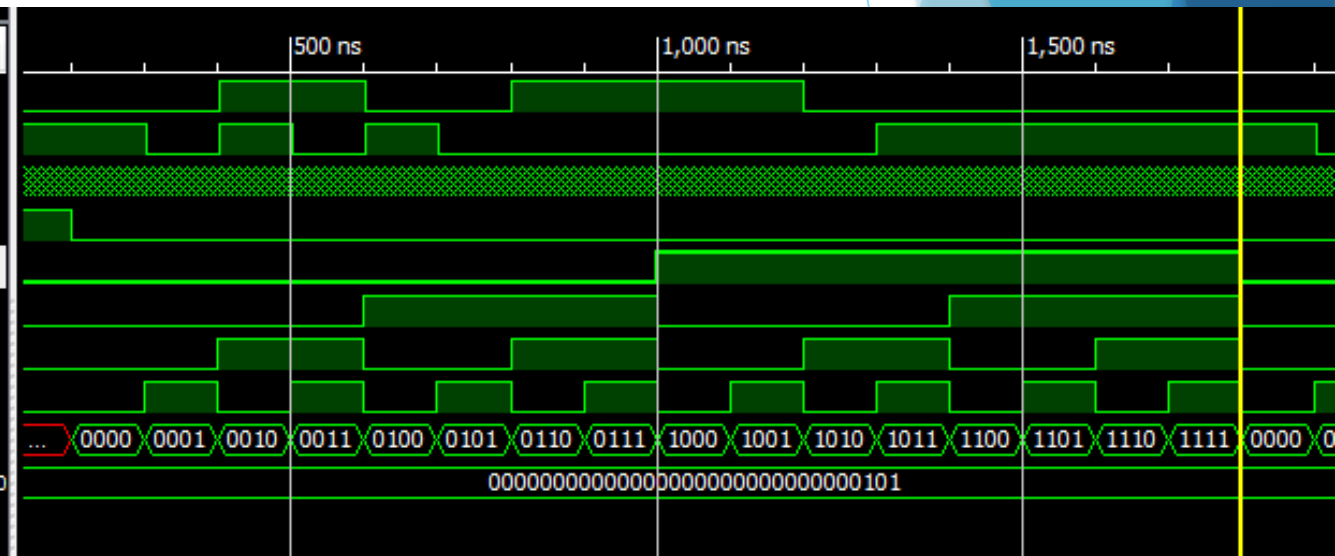
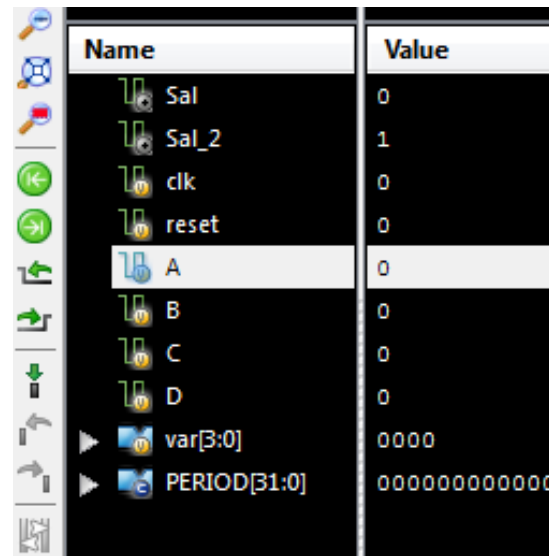
```
assign select_2[0]=B;
assign select_2[1]=A;
```

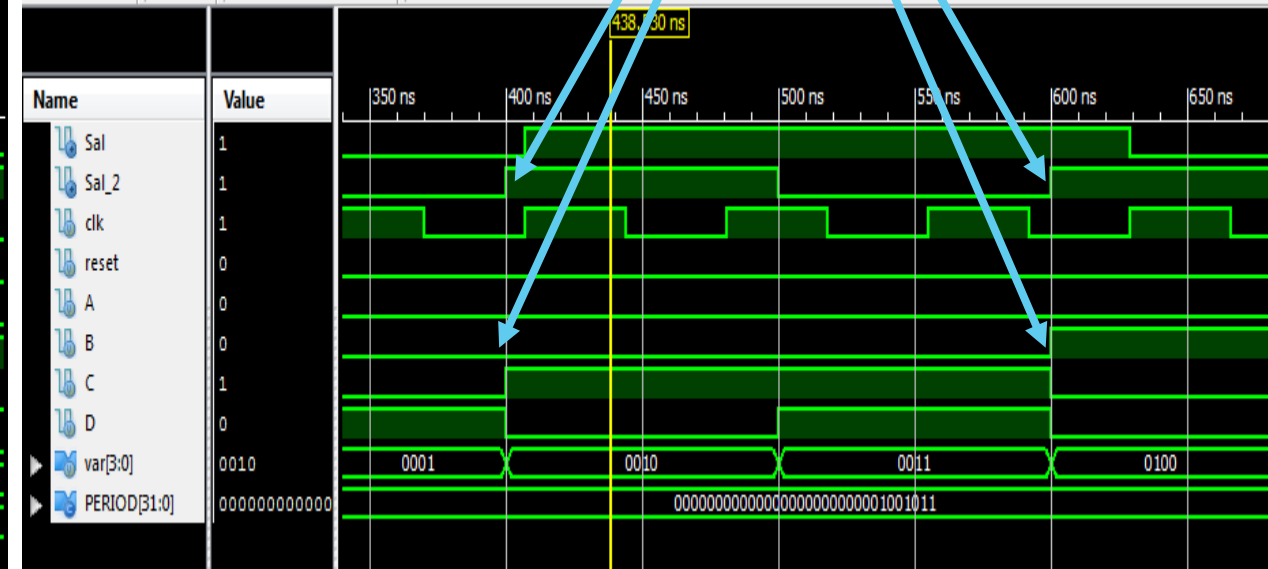
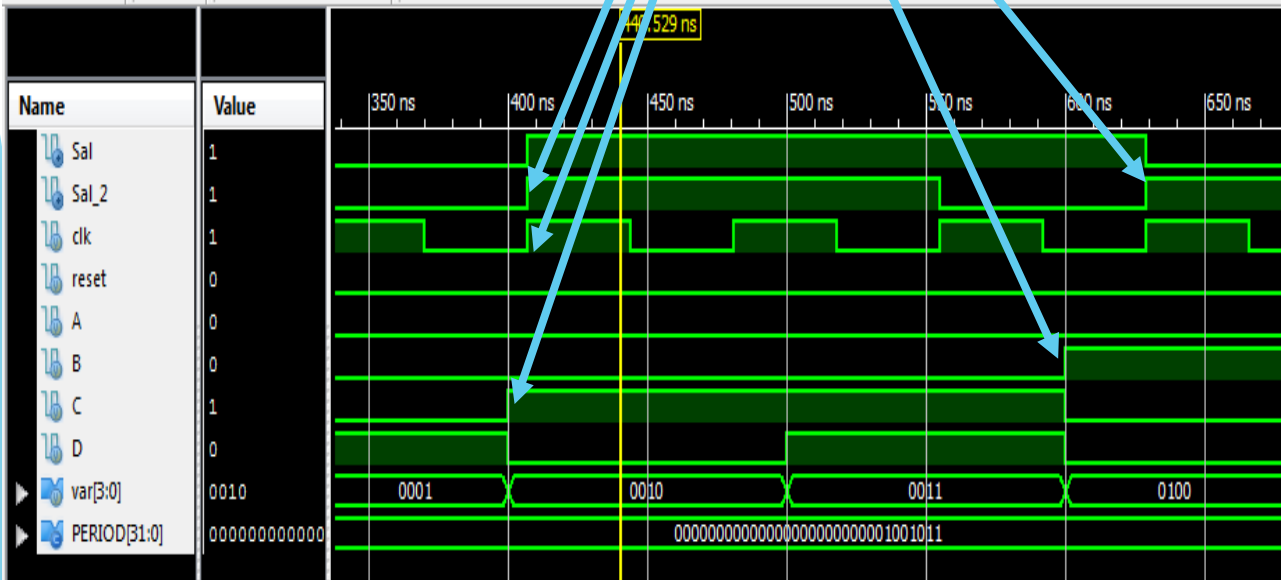
endcase

F=1



	A	B	C	D	F(a)
m_0	0	0	0	0	0
m_1	0	0	0	1	1
m_2	0	0	1	0	0
m_3	0	0	1	1	1
m_4	0	1	0	0	1
m_5	0	1	0	1	0
m_6	0	1	1	0	0
m_7	0	1	1	1	0
m_8	1	0	0	0	0
m_9	1	0	0	1	0
m_{10}	1	0	1	0	0
m_{11}	1	0	1	1	1
m_{12}	1	1	0	0	1
m_{13}	1	1	0	1	1
m_{14}	1	1	1	0	1
m_{15}	1	1	1	1	1





$$F1(ABCD) = \sum 0,2,5,8,10,14$$

$$F2(ABCD) = \prod 2,6,11$$

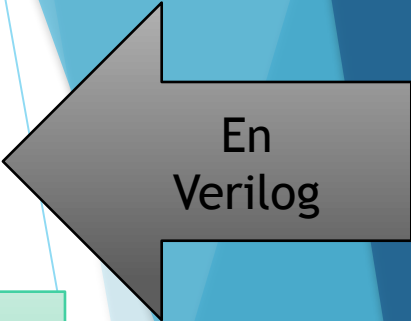
Tarea:

Implementar las siguientes funciones booleana con multiplexor utilizando como herramienta de descripción de hardware al lenguaje Verilog.

Realizar F1 y F2 con multiplexor 4 A 1

Realizar F1 y F2 con multiplexor 8 A 1

Cada proyecto debe tener ambas salidas resueltas con la misma entidad

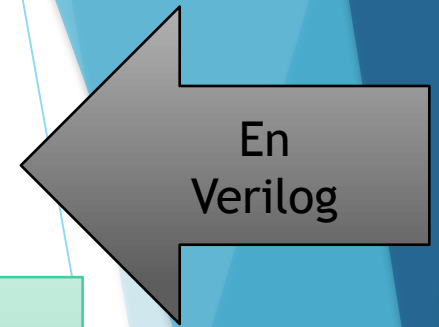


En
Verilog

[Simular Proteus](#)

$$F1(ABC) = \sum 1,2,3,6$$

$$F2(ABC) = \prod 0,1,3,4$$



Tarea:

Implementar las siguientes funciones booleana con multiplexor utilizando como herramienta de descripción de hardware al lenguaje Verilog.

Realizar F1 y F2 con multiplexor 4 A 1

Cada proyecto debe tener ambas salidas resueltas con la misma entidad y con un único mux.

Nota: El multiplexor debe ser síncrono y debe tener reset donde toda las salidas, se ponen a '0'

4-35. Implementar las siguientes funciones booleanas con multiplexores 4 a 1 y compuertas.

(a) $F_1(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$

(b) $F_2(A, B, C, D) = \Sigma(1, 2, 5, 7, 8, 10, 11, 13, 15)$

	A	B	C	D	F(a)
m ₀	0	0	0	0	1
m ₁	0	0	0	1	1
m ₂	0	0	1	0	0
m ₃	0	0	1	1	1
m ₄	0	1	0	0	0
m ₅	0	1	0	1	1
m ₆	0	1	1	0	1
m ₇	0	1	1	1	0
m ₈	1	0	0	0	1
m ₉	1	0	0	1	1
m ₁₀	1	0	1	0	0
m ₁₁	1	0	1	1	1
m ₁₂	1	1	0	0	1
m ₁₃	1	1	0	1	1
m ₁₄	1	1	1	0	0
m ₁₅	1	1	1	1	0

		D	
		0	1
C	0	1	1
	1	0	1

$$F = D + C'$$

		D	
		0	1
C	0	0	1
	1	1	0

$$F = \bar{C}.D + C.D'$$

		D	
		0	1
C	0	1	1
	1	0	1

$$F = D + C'$$

		D	
		0	1
C	0	1	1
	1	0	0

$$F = C'$$

En
Verilog